



Федеральное государственное бюджетное образовательное учреждение высшего образования
“РОССИЙСКАЯ АКАДЕМИЯ НАРОДНОГО ХОЗЯЙСТВА
И ГОСУДАРСТВЕННОЙ СЛУЖБЫ при ПРЕЗИДЕНТЕ РОССИЙСКОЙ ФЕДЕРАЦИИ”

Институт экономики, математики и информационных технологий
Факультет информационных технологий и анализа данных
Кафедра системного анализа и информатики

Машинное обучение для Интернета вещей и мобильных устройств

Лекция 2

Направление подготовки: 09.04.03 Прикладная информатика
Направленность (профиль) образовательной программы:
«Анализ данных и искусственный интеллект»
(очная форма обучения)

Преподаватель: Бочкарёв Пётр Владимирович

Москва
2026

Введение в компьютерное зрение

Компьютерное зрение — это область, занимающаяся тем, как компьютеры могут быть обучены извлекать информацию из изображений и видео, а также интерпретировать визуальные данные. Эта технология используется в различных приложениях, начиная от распознавания лиц и заканчивая анализом медицинских изображений. Компьютерное зрение позволяет системам "видеть" и понимать окружающий мир, что открывает новые горизонты для автоматизации и повышения эффективности.



Компьютерное зрение в IoT в различных областях:

- **Системы видеонаблюдения.** Алгоритмы компьютерного зрения анализируют видео в реальном времени, позволяя обнаруживать аномалии, такие как вторжения или подозрительное поведение.
- **Автономные транспортные средства.** Использование камер и датчиков для восприятия окружающей среды, включая распознавание дорожных знаков, пешеходов и других транспортных средств.
- **Системы безопасности.** Распознавание лиц и обнаружение вторжений обеспечивают повышенную безопасность в общественных местах и частных домах.
- **Анализ поведения.** Системы, отслеживающие движение людей и объектов, помогают оптимизировать торговые площади, улучшая покупательский опыт и увеличивая доходы бизнеса.



OpenCV

- **OpenCV** (Open Source Computer Vision Library) — это библиотека, предназначенная для обработки изображений и видео, а также для разработки приложений в области компьютерного зрения и машинного обучения
- **OpenCV** предоставляет инструменты для выполнения задач, таких как распознавание объектов, обработка изображений, анализ видео и многое другое. Библиотека поддерживает множество языков программирования, включая Python, C++ и Java
- Установка в Python **!pip install opencv-python**

Чтение и отображение изображений

- Изображение в контексте компьютера представляет собой массив данных, который хранит информацию о визуальных элементах. Основные аспекты изображения с точки зрения компьютера:
 - Изображение состоит из маленьких единиц, называемых пикселями. Каждый пиксель содержит информацию о цвете (например, красный, зеленый и синий в RGB) и яркости(для черно-белых изображениях пиксели представляют собой градации серого)
 - Изображения могут быть представлены в различных цветовых моделях. Например, RGB, CMYK, HSV
 - Размер изображения определяется количеством пикселей по ширине и высоте (например, 1920x1080). Разрешение указывает на количество пикселей на единицу площади и влияет на четкость и детализацию изображения

Чтение и отображение изображений

- Функция **cv2.imread(filename, flags)** используется для загрузки изображения из файла
- Параметры:
 - **filename**: путь к файлу изображения
 - **flags**: режим чтения изображения:
 - **cv2.IMREAD_COLOR**: загружает цветное изображение (по умолчанию).
 - **cv2.IMREAD_GRAYSCALE**: загружает изображение в градациях серого.

Отображение изображений

- Функция `cv2.imshow(window_name, image)` используется для отображения изображения в отдельном окне
- Параметры:
 - **window_name**: окно, в котором будет отображаться изображение
 - **image**: изображение, которое нужно отобразить
- Использование OpenCV в Google Colab:
 - команда для установки пакета: `from google.colab.patches import cv2_imshow`
 - Функция `cv2_imshow(image)`
- Сохранение изображений выполняется с помощью функции `cv2.imwrite()`, которая сохраняет изображение в указанном формате и по заданному пути

Основные операции с изображениями

- Изменение размера изображений
- Обрезка изображений
- Поворот изображений

Изменение размера изображений

- Для изменения размера изображений используется функция **cv2.resize(image, (width, height))**, которая принимает новые размеры (ширину и высоту) и может масштабировать изображение, сохраняя его пропорции
- Часто используется для подготовки изображений к обработке, так как многие алгоритмы требуют, чтобы входные изображения имели одинаковый размер

Обрезка изображений

- Обрезка изображений позволяет выделить интересующую область, определяемую координатами верхнего левого угла и размерами (шириной и высотой). Это позволяет сосредоточиться на конкретных объектах или регионах изображения
- Для обрезки изображений используется индексирование NumPy, т.е. формат `image[y:y+h, x:x+w]`, где:
 - `x`: координата начала по оси X.
 - `y`: координата начала по оси Y.
 - `w`: ширина обрезаемой области.
 - `h`: высота обрезаемой области.

Например, для обрезки области 100x100 пикселей, начиная с координат (50, 50), код будет выглядеть так: `cropped_image = image[50:150, 50:150]`.

Поворот изображений

- Поворот изображений реализуется с использованием матрицы преобразования, созданной с помощью функции **cv2.getRotationMatrix2D()**. Данная матрица описывает параметры поворота, такие как угол и центр поворота. Функция **cv2.warpAffine()** применяет это преобразование к изображению
- Поворот используют для выравнивания изображений или изменения их ориентации
- **cv2.getRotationMatrix2D(center, angle, scale):**
 - center: кортеж (x, y), указывающий центр поворота изображения
 - angle: угол поворота в градусах (положительные значения означают поворот против часовой стрелки)
 - scale: Множитель масштаба (обычно 1.0, если не требуется масштабирование)
- **cv2.warpAffine(src, M, dsize)**
 - src: исходное изображение (NumPy массив)
 - M: матрица преобразования, полученная из cv2.getRotationMatrix2D()
 - dsize: кортеж (width, height), определяющий размер выходного изображения.

Преобразование цветового пространства

- Конвертация в серый цвет
- Другие цветовые пространства
- Применение цветowych фильтров

Конвертация в серый цвет

- Преобразование цветного изображения в оттенки серого выполняется с помощью функции **cv2.cvtColor(src, code)**
- Используется для упрощения анализа изображений и уменьшения вычислительных затрат, так как работа с одним каналом (оттенками серого) требует меньше ресурсов, чем с тремя каналами (RGB)
- Параметры:
 - `src`: исходное изображение (NumPy массив).
 - `code`: код, определяющий тип преобразования. Для конвертации в серый цвет используется `cv2.COLOR_BGR2GRAY` (или `cv2.COLOR_RGB2GRAY`, в зависимости от порядка цветовых каналов)

Другие цветовые пространства

- Для преобразование изображения в другие цветовые пространства также используется функция **cv2.cvtColor(src, code)**
 - **BGR (Blue, Green, Red)** : стандартное цветовое пространство. Изображения хранятся в формате BGR, где каждый пиксель представлен тремя значениями, соответствующими интенсивности синего, зеленого и красного цветов. Часто требуется преобразования из BGR в RGB (для отображения в других библиотеках) (cv2.COLOR_BGR2RGB)
 - **HSV (Hue, Saturation, Value)** : цветовое пространство, которое разделяет цвет (оттенок) от яркости. Hue (оттенок) определяет цвет, Saturation (насыщенность) — интенсивность цвета, Value (яркость) — светлоту. Это пространство удобно для задач, связанных с сегментацией по цвету. Код для cvtColor: cv2.COLOR_BGR2HSV
 - **LAB (CIE Lab)** : цветовое пространство, которое пытается быть более близким к восприятию цвета человеком. Оно разделяет яркость (L) от цветовых компонентов (a и b), что делает его полезным для задач, связанных с цветовой коррекцией и анализом. Код для cvtColor: cv2.COLOR_BGR2Lab
 - **YUV** : цветовое пространство, которое разделяет яркость (Y) и цветовые компоненты (U и V). Это пространство часто используется в видеообработке и сжатии, так как позволяет более эффективно кодировать изображения. Код для cvtColor: cv2.COLOR_BGR2YUV
 - **YCrCb** : цветовое пространство разделяет яркость и цветовые компоненты, но используется в основном в цифровом видео и телевидении. Позволяет более эффективно сжимать данные, так как человеческий глаз менее чувствителен к изменениям в цвете, чем в яркости. Код для cvtColor: cv2.COLOR_BGR2YCrCb

Применение цветowych фильтров

- Цветовые фильтры позволяют выделять определенные цвета на изображении, что полезно для задач сегментации и распознавания объектов
- Функция **cv2.inRange()** создает маску, которая выделяет пиксели, попадающие в заданный диапазон цветowych значений
- Параметры:
 - src: исходное изображение
 - lowerb: нижняя граница диапазона цвета
 - upperb: верхняя граница диапазона цвета

Пример:

- `hsv_image = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)`
- `mask = cv2.inRange(hsv_image, lower_bound, upper_bound)`

Фильтрация изображений

- **Фильтрация изображений** — процесс обработки изображений, который позволяет улучшить качество изображений, уменьшить шум, выделить детали и подготовить изображения для дальнейшего анализа
- **Варианты фильтрации:**
 - Гауссовское размытие
 - Медианная фильтрация
 - Фильтрация по среднему значению
 - Билатеральное размытие

Гауссовское размытие

- Гауссовское размытие используется для уменьшения шума и деталей изображения. Оно основано на применении гауссовского ядра, которое придает больший вес пикселям, находящимся ближе к центру, и меньший — пикселям, находящимся дальше.
- Метод часто используется перед применением алгоритмов обнаружения краев, так как он помогает сгладить изображение и уменьшить влияние шума.

Гауссовское размытие

- **cv2.GaussianBlur(src, ksize, sigmaX, sigmaY)**
 - src: исходное изображение (NumPy массив), которое будет подвергнуто размытия.
 - ksize: размер ядра, который должен быть нечетным числом (например, 3, 5, 7)
 - sigmaX: стандартное отклонение по оси X. Если установлено в 0, стандартное отклонение будет вычислено автоматически на основе размера ядра. Это значение определяет, насколько сильно будет происходить размытие
 - sigmaY: стандартное отклонение по оси Y (опционально)

Медианная фильтрация

- Медианная фильтрация эффективно удаляет шум "соль и перец" и сохраняет края. Этот метод часто используется в задачах, где требуется сохранить структуру изображения, несмотря на наличие шума
- **cv2.medianBlur()**
 - **src:** исходное изображение (NumPy массив), которое будет подвергнуто медианной фильтрации.
 - **ksize:** размер ядра, который должен быть нечетным числом (например, 3, 5, 7). Размер ядра определяет область, в которой будет вычисляться медиана.

Фильтрация по среднему значению

- Метод заменяет значение каждого пикселя на среднее значение пикселей в его окрестности. Он используется для сглаживания изображений и уменьшения шума.
- Фильтрация по среднему значению часто используется в задачах, где требуется общее сглаживание изображения
- **cv2.blur()**
 - **src:** исходное изображение (NumPy массив), которое будет подвергнуто фильтрации
 - **ksize:** размер ядра, который должен быть положительным и нечетным числом (например, 3, 5, 7). Размер ядра определяет область, в которой будет вычисляться среднее значение

Билатеральное размытие

- Метод фильтрации изображений, который позволяет сглаживать изображение, сохраняя при этом его края.
- В отличие от обычных методов размытия, билатеральное размытие учитывает как пространственные, так и цветовые характеристики пикселей.
- Пиксели, находящиеся близко друг к другу и имеющие схожие значения цвета, будут иметь больший вес при вычислении нового значения пикселя, что позволяет эффективно сохранять детали и контуры
- Билатеральное размытие часто используется в задачах, где необходимо уменьшить шум изображения, сохраняя при этом важные детали, такие как края

Билатеральное размытие

- **cv2.bilateralFilter()**
 - **src:** исходное изображение, к которому будет применяться билатеральное размытие.
 - **d:** диаметр области, в которой будет применяться фильтр. Это значение определяет, сколько соседних пикселей будет учитываться при вычислении нового значения. Чем больше значение, тем больше область, но это также может увеличить вычислительные затраты
 - **sigmaColor:** стандартное отклонение в цветовом пространстве. Это значение определяет, насколько сильно различие в цвете между пикселями влияет на их вес. Чем больше значение, тем более "размытыми" будут цвета, что может привести к более сильному сглаживанию
 - **sigmaSpace:** стандартное отклонение в пространственном пространстве. Это значение определяет, насколько сильно расстояние между пикселями влияет на их вес. Чем больше значение, тем больше пикселей будет учитываться, даже если они находятся далеко друг от друга

Бинаризация

- **Бинаризация** — процесс преобразования изображения в черно-белый формат, где каждый пиксель принимает одно из двух значений: 0 (черный) или 255 (белый). Этот процесс позволяет выделить объекты на изображении, упрощая дальнейшую обработку и анализ.
- Бинаризация часто используется в задачах, связанных с распознаванием объектов, анализом текстов и сегментацией изображений
- Методы бинаризации:
 - Фиксированная бинаризация
 - Алгоритм Оцу
 - Адаптивная бинаризация

Фиксированная бинаризация

- Метод использует фиксированный порог для преобразования изображения. Если значение пикселя выше порога, он становится белым (255), если ниже — черным (0).
- **cv2.threshold():**
 - **src:** исходное изображение (обычно в оттенках серого).
 - **thresh:** значение порога, которое используется для бинаризации.
 - **maxval:** максимальное значение, присваиваемое пикселям, превышающим порог (обычно 255).
 - **type:** тип пороговой обработки. Например, `cv2.THRESH_BINARY` для стандартной бинаризации или `cv2.THRESH_BINARY_INV` для инверсии.

Алгоритм Оцу

- Алгоритм Оцу автоматически определяет оптимальный порог для бинаризации, минимизируя внутриклассовую дисперсию. Это особенно полезно для изображений с неоднородным освещением
- **cv2.threshold()**
 - Аналогичны фиксированной бинаризации,
 - В качестве типа используется **cv2.THRESH_BINARY + cv2.THRESH_OTSU**.

Адаптивная бинаризация

- Метод вычисляет порог для каждого пикселя на основе значений его соседей. Это позволяет лучше справляться с изображениями с неравномерным освещением.
- **cv2.adaptiveThreshold()**
 - src: исходное изображение
 - maxValue: максимальное значение, присваиваемое пикселям, превышающим порог (обычно 255)
 - adaptiveMethod: метод адаптивного вычисления порога. Например, **cv2.ADAPTIVE_THRESH_MEAN_C** или **cv2.ADAPTIVE_THRESH_GAUSSIAN_C**
 - thresholdType: тип пороговой обработки (обычно cv2.THRESH_BINARY)
 - **blockSize**: размер области, используемой для вычисления порога (должен быть нечетным числом).
 - **C**: константа, вычитаемая из среднего или взвешенного среднего, чтобы настроить порог.

Морфологические операции

- **Морфологические операции** — это набор методов обработки изображений, которые работают на основе формы объектов в изображении. Эти операции особенно полезны для обработки бинарных изображений, но могут также применяться к изображениям в оттенках серого. Основные морфологические операции включают эрозию, дилатацию, открытие и закрытие.
- Методы морфологически операций:
 - Эрозия
 - Дилатация
 - Открытие\Закрытие
 - Морфологический градиент
 - Верхняя и нижняя шляпа

Эрозия

- Эрозия уменьшает размеры объектов в бинарном изображении. Она работает, "сжимая" объекты, удаляя пиксели на границах. Это помогает удалить шум и мелкие детали, которые не соответствуют заданной форме.
- **cv2.erode()**
 - **src:** исходное бинарное изображение.
 - **kernel:** ядро (структурный элемент), определяющее форму, с которой будет выполняться операция. Обычно создается с помощью `cv2.getStructuringElement()`.
 - **iterations:** количество итераций операции (по умолчанию 1).

Дилатация

- Дилатация увеличивает размеры объектов в бинарном изображении. Она работает, "расширяя" объекты, добавляя пиксели к границам. Это полезно для заполнения пробелов и соединения разрозненных частей объектов.
- **cv2.dilate()**
- Параметры:
 - **src**: исходное бинарное изображение.
 - **kernel**: ядро (структурный элемент), определяющее форму, с которой будет выполняться операция.
 - **iterations**: количество итераций операции (по умолчанию 1).

Открытие

- Открытие — это комбинация эрозии, за которой следует дилатация. Эта операция помогает удалить мелкие шумы и сохраняет форму и размер больших объектов. Открытие полезно для удаления небольших объектов из изображения.
- **cv2.morphologyEx()**
 - src: исходное бинарное изображение
 - op: тип операции, в данном случае cv2.MORPH_OPEN
 - kernel: ядро (структурный элемент)

Закрытие

- Закрытие — это комбинация дилатации, за которой следует эрозия. Эта операция помогает заполнить небольшие отверстия и пробелы в объектах, сохраняя при этом их форму. Закрытие полезно для соединения разрозненных частей объектов.
- **cv2.morphologyEx()**
 - src: исходное бинарное изображение
 - op: тип операции, в данном случае cv2.MORPH_CLOSE
 - kernel: ядро (структурный элемент)

Морфологический градиент

- Морфологический градиент вычисляет разницу между дилатацией и эрозией изображения. Это позволяет выделить границы объектов, подчеркивая изменения в форме.
- `cv2.morphologyEx()`
 - `src`: исходное бинарное изображение.
 - `op`: тип операции, в данном случае `cv2.MORPH_GRADIENT`.
 - `kernel`: ядро (структурный элемент).

Верхняя и нижняя шляпа

- **Верхняя шляпа (Top Hat)** выделяет детали, которые были удалены при открытии. Это полезно для выделения мелких объектов на темном фоне.
- **Нижняя шляпа (Black Hat)** выделяет детали, которые были удалены при закрытии. Это полезно для выделения мелких объектов на светлом фоне.
- **`cv2.morphologyEx()`**
 - `src`: исходное бинарное изображение.
 - `op`: тип операции, `cv2.MORPH_TOPHAT` для верхней шляпы и `cv2.MORPH_BLACKHAT` для нижней шляпы.
 - `kernel`: ядро (структурный элемент).

Структурный элемент

- В контексте морфологических операций, ядро (или структурный элемент) — это матрица, которая определяет форму и размер области, используемой для обработки изображения. Ядро используется для выполнения операций, таких как эрозия и дилатация, которые являются основными компонентами морфологического градиента.
- Ядро определяет, какие пиксели будут учитываться при выполнении морфологических операций. Например, при применении морфологического градиента, ядро используется для вычисления разницы между дилатацией и эрозией изображения. Это позволяет выделить границы объектов, подчеркивая изменения в интенсивности пикселей.
- Ядро может иметь различные формы и размеры, в зависимости от задачи. Наиболее распространенные формы включают:
 - Квадратное ядро: используется для обработки изображений с равномерными объектами.
 - Круглое ядро: лучше подходит для обработки изображений с округлыми формами.
 - Эллиптическое ядро: используется для более сложных форм.

Структурный элемент

- В OpenCV ядро можно создать с помощью функции **cv2.getStructuringElement()**, где можно указать форму и размер ядра. Например:
 - Создание квадратного ядра размером 5x5:
 - `kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))`
 - Создание круглого ядра размером 5:
 - `kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))`
 - Создание крестовидного ядра размером 5x5:
 - `kernel = cv2.getStructuringElement(cv2.MORPH_CROSS, (5, 5))`

Алгоритм Кэнни

- **cv2.Canny()**
 - **Image:** NumPy массив (обычно в оттенках серого). Исходное изображение, к которому будет применяться алгоритм Кэнни. Обычно перед использованием алгоритма изображение преобразуется в оттенки серого и сглаживается.
 - **Threshold1** тип: Целое число. Первый порог для определения сильных границ. Пиксели с градиентом выше этого порога будут считаться сильными границами. Этот параметр влияет на чувствительность алгоритма к изменениям в интенсивности.
 - **Threshold2** тип: Целое число. Второй порог для определения слабых границ. Пиксели с градиентом ниже этого порога будут игнорироваться. Пиксели, значения которых находятся между threshold1 и threshold2, будут считаться слабыми границами и могут быть сохранены, если они соединены с сильными границами.
 - **apertureSize** (опционально) Тип: Целое число. Размер апертуры для оператора Собеля, используемого для вычисления градиента. Возможные значения: 1, 3, 5 или 7. По умолчанию значение равно 3. Увеличение этого параметра может улучшить обнаружение границ, но также может привести к потере деталей.
 - **L2gradient** (опционально) Тип: Булевый (True/False). Если установлено в True, используется более точный метод для вычисления градиента, основанный на L2-норме. Если False, используется L1-норма. По умолчанию значение равно False

Выделение контуров

- Функция **cv2.findContours()** используется для обнаружения контуров в бинарном изображении. Контурные представляют собой кривые, соединяющие все непрерывные точки (по границе), имеющие одинаковый цвет или интенсивность. Эта функция возвращает список контуров и иерархию контуров.
- Параметры:
 - **image**: Исходное бинарное изображение (NumPy массив), в котором будут искаться контуры. Обычно это изображение, полученное после бинаризации.
 - **mode**: Режим поиска контуров. Возможные значения:
 - **cv2.RETR_EXTERNAL**: Извлекает только внешние контуры.
 - **cv2.RETR_LIST**: Извлекает все контуры, не устанавливая иерархию.
 - **cv2.RETR_TREE**: Извлекает все контуры и создает иерархию.
 - **method**: Метод аппроксимации контуров. Возможные значения:
 - **cv2.CHAIN_APPROX_SIMPLE**: Упрощает контуры, сохраняя только конечные точки.
 - **cv2.CHAIN_APPROX_NONE**: Сохраняет все точки контура.
- Возвращаемые значения:
 - **contours**: Список найденных контуров.
 - **hierarchy**: Иерархия контуров (опционально).

Рисование контуров

- Визуализация контуров на изображении осуществляется с помощью `cv2.drawContours()`, что позволяет проверять работу алгоритмов и анализировать результаты.
- Рисование контуров помогает лучше понять, как алгоритмы обрабатывают изображения и какие области они выделяют.
- Параметры:
 - **image**: исходное изображение (NumPy массив), на котором будут рисоваться контуры.
 - **contours**: список контуров, полученных с помощью `cv2.findContours()`.
 - **contourIdx**: индекс контура, который нужно нарисовать. Если установлен в -1, рисуются все контуры.
 - **color**: цвет контуров в формате BGR (например, (255, 0, 0) для красного).
 - **thickness**: толщина линий контуров. Если установлено в -1, контуры будут заполнены.

Заключение

OpenCV предоставляет мощные инструменты для обработки изображений и видео, включая фильтрацию, обнаружение объектов и работу с цветами. Библиотека широко используется в различных областях, таких как медицина, безопасность и автономные системы.



ПРЕЗИДЕНТСКАЯ
АКАДЕМИЯ

Федеральное государственное бюджетное образовательное учреждение высшего образования
“РОССИЙСКАЯ АКАДЕМИЯ НАРОДНОГО ХОЗЯЙСТВА
И ГОСУДАРСТВЕННОЙ СЛУЖБЫ при ПРЕЗИДЕНТЕ РОССИЙСКОЙ ФЕДЕРАЦИИ”

Институт экономики, математики и информационных технологий
Факультет информационных технологий и анализа данных
Кафедра системного анализа и информатики

2025

Машинное обучение для Интернета вещей и мобильных устройств

Лекция 2

Направление подготовки: 09.04.03 Прикладная информатика
Направленность (профиль) образовательной программы:
«Анализ данных и искусственный интеллект»
(очная форма обучения)

Преподаватель: Бочкарёв Пётр Владимирович

РАНХиГС

Москва

Москва
2026